

CS2102: Database Systems
Audit
Summary

Sim Ray En Ryan

December 14, 2025

Contents

1	L01: Overview of Relational Model	6
1.1	Course Objectives and Components	6
1.1.1	Learning Objectives	6
1.1.2	Theory and Practice Components	6
1.1.3	Learning Outcomes by Component	6
1.1.4	Prerequisites	6
1.2	Database Management Systems (DBMS)	7
1.2.1	What is a Database?	7
1.2.2	Challenges in Data-Intensive Applications (Why DBMS?)	7
1.2.3	Data Management with DBMS	7
1.3	Transactions and ACID Properties	7
1.3.1	Definition	7
1.3.2	ACID Properties	7
1.3.3	Serializability	8
1.4	DBMS Architecture and Data Independence	8
1.4.1	3-Tier Architecture (Levels of Data Abstraction)	8
1.4.2	Data Independence	8
1.5	Relational Model	8
1.5.1	History	8
1.5.2	The Relational Model Concepts	8
1.6	Data Integrity (Integrity Constraints)	9
1.6.1	Definition	9
1.6.2	Kinds of Integrity Constraints	9
1.6.3	Key Constraints	9
1.6.4	Foreign Key Constraints (Referential Integrity)	9
2	L02: SQL Data Definition Language (DDL)	10
2.1	SQL Overview	10
2.1.1	What is SQL?	10
2.1.2	Kinds of Commands	10
2.2	Creating, Inserting, Deleting, and Updating	10
2.2.1	CREATE TABLE	10
2.2.2	INSERT INTO	11
2.2.3	DELETE FROM	11
2.2.4	UPDATE	11
2.3	Three-Valued Logic in SQL	12
2.3.1	Logical Operations	12
2.3.2	Relational and Arithmetic Operations	12
2.3.3	Handling NULL as Values	12
2.4	Integrity Constraints in SQL	12
2.4.1	Preliminary	12

2.4.2	Specific Constraints	12
2.4.3	Foreign Key Actions	13
2.4.4	General Check Constraint	13
2.5	Modifying Database (ALTER and DROP)	13
2.5.1	ALTER TABLE	13
2.5.2	DROP TABLE	14
2.6	Deferrable Constraints	14
2.6.1	Motivation	14
2.6.2	Deferment	14
3	L03: Entity-Relationship (ER) Model	14
3.1	Database Design Process	14
3.2	ER Model Concepts and Elements	15
3.2.1	ER Model	15
3.2.2	Core Concepts	15
3.2.3	Attribute Kinds	15
3.2.4	Relationship Degrees	15
3.3	Relationship Constraints (Look Here Notation)	16
3.3.1	Cardinality Constraints (Upper Bound)	16
3.3.2	Participation Constraints (Lower Bound)	16
3.3.3	Dependency Constraints (Weak Entity Sets)	16
3.4	Relational Mapping (ER Diagram to Schema)	16
3.4.1	Entity Set Mapping	16
3.4.2	Relationship Set Mapping (Binary)	17
3.5	Extended Notations	18
3.5.1	ISA Hierarchies (Generalization/Specialization)	18
3.5.2	Aggregation	18
4	L04: SQL (Part 1)	18
4.1	Basic SQL Queries	18
4.1.1	SELECT-FROM-WHERE Structure	18
4.1.2	Conceptual Evaluation Steps	19
4.1.3	SELECT Clause Details	19
4.1.4	WHERE Clause Details	19
4.2	Set Operations	19
4.3	Join Operations	20
4.3.1	Cartesian Join (Cross Product)	20
4.3.2	Inner Join	20
4.3.3	Natural Join ($\bowtie$)	20
4.3.4	Outer Join	20
4.4	Composing Queries with Subqueries	21
4.4.1	Subqueries	21
4.4.2	Scalar Subquery	21

4.4.3	Set Membership ([NOT] IN)	21
4.4.4	Existence ([NOT] EXISTS)	21
4.4.5	Quantification (ANY / ALL)	22
4.4.6	Scoping Rules	22
4.5	Ordering	22
4.5.1	ORDER BY	22
4.5.2	LIMIT & OFFSET	22
5	L05: SQL (Part 2)	22
5.1	Common SQL Constructs	22
5.1.1	Aggregation Functions	22
5.1.2	Grouping (GROUP BY)	23
5.1.3	Filtering Groups (HAVING)	23
5.1.4	Conditional Expressions	23
5.2	Structuring Queries	24
5.2.1	Table Expressions (WITH)	24
5.2.2	Views (CREATE VIEW)	24
5.3	Extended Concepts	24
5.3.1	Universal Quantification ($\forall$) in SQL	24
5.3.2	Recursive Queries	25
6	L06: Relational Algebra (RA)	25
6.1	Introduction to Relational Algebra	25
6.1.1	Definition	25
6.1.2	Why Relational Algebra?	25
6.1.3	Closure Property	25
6.2	Relational Algebra Operators	26
6.2.1	Unary Operators	26
6.2.2	Binary Operators	26
6.3	Complex Expressions and Equivalence	27
6.3.1	Expression Tree	27
6.3.2	Equivalence and Isomorphism	27
6.3.3	Equivalence Properties (Optimization Rules)	27
7	L07: Programming with SQL: Functions, Procedures, and Cursors	28
7.1	Accessing Database from Host Languages	28
7.1.1	Mixing Host Language with SQL (Statement-Level Interface - SLI)	28
7.1.2	Call-Level Interface (CLI)	28
7.1.3	SQL Injection Attacks	28
7.2	Functions and Procedures in SQL (PL/pgSQL)	29
7.2.1	Motivation	29
7.2.2	SQL Functions (CREATE OR REPLACE FUNCTION)	29
7.2.3	SQL Procedures (CREATE OR REPLACE PROCEDURE)	29

7.2.4	Control Structures	29
7.2.5	Cursors	29
8	L08: Programming with SQL: Triggers and Normal Forms (Part 1)	30
8.1	Triggers	30
8.1.1	Motivation	30
8.1.2	Trigger Definition (<code>CREATE TRIGGER</code>)	30
8.1.3	Trigger Function (<code>CREATE OR REPLACE FUNCTION ...RETURNS TRIGGER</code>)	30
8.1.4	Trigger Timing	30
8.1.5	Return Values	31
8.1.6	Trigger Levels	31
8.1.7	Trigger Condition (<code>WHEN</code>)	31
8.1.8	Deferred Trigger (<code>CONSTRAINT TRIGGER</code>)	31
8.2	Normal Forms (NF) and Functional Dependencies (FD)	32
8.2.1	Motivation	32
8.2.2	Functional Dependency (FD)	32
8.2.3	FD Reasoning (Armstrong's Axioms)	32
8.2.4	Closure of an Attribute Set (X^+)	32
8.2.5	Keys and Superkeys via FDs	33
9	L09: Normal Forms (Part 2)	33
9.1	BCNF (Boyce-Codd Normal Form)	33
9.1.1	Non-trivial and Decomposed FDs	33
9.1.2	BCNF Definition	33
9.1.3	Simplified BCNF Check (Using Closure)	33
9.1.4	BCNF Decomposition	34
9.1.5	Properties of BCNF	34
9.2	Third Normal Form (3NF)	34
9.2.1	3NF Definition	34
9.2.2	BCNF vs. 3NF	35
9.2.3	3NF Check (Using Keys and Prime Attributes)	35
9.2.4	3NF Decomposition (Synthesis Algorithm)	35
9.2.5	Minimal Basis	35
9.2.6	Properties of 3NF	36

1 L01: Overview of Relational Model

1.1 Course Objectives and Components

1.1.1 Learning Objectives

- The aim of this module is to introduce the fundamental concepts and techniques necessary for the understanding and practice of design and implementation of database applications and of the management of data with relational database management systems.
- The module covers practical and theoretical aspects of design with Entity-Relationship (ER) model, theory of functional dependencies, and normalization decomposition in Third (3NF) and Boyce-Codd Normal Forms (BCNF).
- The module covers practical and theoretical aspects of programming with SQL data definition and manipulation sublanguages as well as relational algebra/calculus.

1.1.2 Theory and Practice Components

- **Theory:** Entity-Relationship Model, Relational Algebra, Functional Dependencies, Normal Forms.
- **Practice:** SQL (DDL, Queries, Procedures, Triggers).

1.1.3 Learning Outcomes by Component

- **Design:** How to design an Entity-Relationship data model to capture data requirements and translate it to a relational database schema.
 - Topics: Entity-Relationship Model, SQL: DDL.
- **Query:** How to use SQL to define schemas and write queries, and how to reason about correctness using relational algebra/calculus.
 - Topics: SQL: Queries, Relational Algebra, Procedures, Triggers.
- **Refinement:** How to refine a relational database schema using schema decompositions to avoid anomalies.
 - Topics: Functional Dependencies, Normal Forms.

1.1.4 Prerequisites

- **Programming:** CS1020, CS2020, CS2030, CS2040 (required for the project component).
- **Discrete Mathematics:** CS1231, MA1100 (required for the second half of the module on functional dependencies).

1.2 Database Management Systems (DBMS)

1.2.1 What is a Database?

- A database is a collection of organized data, information, and records.

1.2.2 Challenges in Data-Intensive Applications (Why DBMS?)

- **Efficiency:** Fast access to information in huge volumes of data.
- **Transactions:** "All-or-nothing" changes to data (e.g., bank transfer).
- **Data Integrity:** Parallel access and changes to data.
- **Recovery:** Fast and reliable handling of failures.
- **Security:** Fine-grained data access rights.
- Using file-based data management for these requirements leads to high development effort, long development times, and a higher risk of errors, as complex, low-level code must be replicated across different programs.

1.2.3 Data Management with DBMS

- DBMS abstracts away the low-level complexities (Indexing, Recovery, Security, etc.) into a central system.
- The application logic operates using a query language, leading to faster application development, increased productivity, and higher stability.

1.3 Transactions and ACID Properties

1.3.1 Definition

- A transaction is a finite sequence of database operations (reads and/or writes) and constitutes the smallest logical unit of work from the application perspective.

1.3.2 ACID Properties

Each transaction T has the following properties:

- **Atomicity:** Either all effects of T are reflected in the database or none.
- **Consistency:** T guarantees to yield the correct state of the database (i.e., satisfies all user-specified constraints).
- **Isolation:** T is isolated from the effect of concurrent transactions.
- **Durability:** After a commit of T , its effects are permanent.

1.3.3 Serializability

- **Equivalent Transaction:** Two executions are equivalent if they have the same effect on the data.
- **Serializability:** A concurrent execution of a set of transactions is serializable if this execution is equivalent to some serial execution of the same set of transactions.
- DBMS supports concurrent executions for performance but enforces serializability to ensure data integrity.

1.4 DBMS Architecture and Data Independence

1.4.1 3-Tier Architecture (Levels of Data Abstraction)

- **External Schema (View Level):** User or group-specific view on the data.
- **Logical Schema (Conceptual Level):** Logical organization of data (e.g., relations/tables, objects, graphs). It is a unified representation of all data and supports both logical and physical data independence.
- **Physical Schema (Internal Level):** Organization of data on disk and in memory (e.g., collections of fields, arrays, records, files, pages, etc.).

1.4.2 Data Independence

- **Logical Data Independence:** Ability to change the logical schema without affecting the external schemas (e.g., adding/deleting/updating attributes).
- **Physical Data Independence:** Representation of data being independent from the physical schema. The physical schema can be changed without affecting the logical schema (e.g., creating indices, new caching strategies).

1.5 Relational Model

1.5.1 History

- Proposed by Edgar F. Codd in 1970.
- Based on Relations (tables with rows and columns).

1.5.2 The Relational Model Concepts

- **Domain:** A set of atomic values (e.g., INT, NUMERIC, TEXT) for an attribute, denoted as $dom(A_i)$. An attribute value v is either in $dom(A_i)$ or is NULL.
- **NULL:** A special value indicating that the value is unknown, invalid, etc.

- **Relation (Relation Instance):** A set of tuples (records). Each instance of a schema $R(A_1, \dots, A_n)$ is a relation which is a subset of $\{(a_1, \dots, a_n) \mid a_i \in \text{dom}(A_i) \cup \{NULL\}\}$.
- **Relational Schema (Relation Definition):** Specifies the attributes (columns) and data constraints (e.g., domain constraints). Notation: $R(A_1 : \text{TYPE}_1, \dots, A_n : \text{TYPE}_n)$.
- **Relational Database Schema:** A set of relation schemas + data constraints.
- **Relational Database:** A collection of tables (relation instances).

1.6 Data Integrity (Integrity Constraints)

1.6.1 Definition

- Integrity constraints are conditions that restrict what constitutes valid data.

1.6.2 Kinds of Integrity Constraints

- **Structural (Application-Independent):**
 - Domain constraints (e.g., cannot store TEXT in INT column).
 - Key constraints.
 - Foreign key constraints.
- **General (Application-Dependent):**
 - Check constraints (e.g., checking salary not being negative).
 - Triggers (the most general kind of constraints).

1.6.3 Key Constraints

- **Superkey:** A subset of attributes that uniquely identifies a tuple in a relation.
- **Key:** A superkey that is also minimal (i.e., no proper subset of the key is a superkey).
Note: minimal $\neq$ minimum.
- **Candidate Keys:** The set of all keys of a given relation.
- **Primary Key:** A selected candidate key. Primary key attributes cannot be NULL. We underline the primary key in the schema notation, e.g., Movies(id : INT, ...).

1.6.4 Foreign Key Constraints (Referential Integrity)

- **Definition:** A foreign key is a subset of attributes of relation R_1 (the **referencing relation**) that refers to the primary key (or any candidate key) of relation R_2 (the **referenced relation**).

- **Requirement:** Each foreign key in R_1 must satisfy one of the following:
 1. Appear as the primary key in R_2 .
 2. Be a NULL value (or a tuple containing at least one NULL value in the foreign key attributes).
- The referencing relation and the referenced relation can be the same.

2 L02: SQL Data Definition Language (DDL)

2.1 SQL Overview

2.1.1 What is SQL?

- **Structured Query Language.** The de-facto standard language to "talk" to Relational Database Management Systems (RDBMS).
- Developed by Donald D. Chamberlin and Raymond F. Boyce (IBM Research, 1974).
- A **declarative language**: focus on *what* to compute, instead of *how* to compute.
- Based on **multi-set/bag semantics** (may contain duplicate elements).
- **SQL Standard:** First standard is SQL-86; current is SQL-2023. Many RDBMS implement their own "flavor" (e.g., PostgreSQL has plpgsql).

2.1.2 Kinds of Commands

- **DDL (Data Definition Language):** CREATE, ALTER, DROP, RENAME, TRUNCATE.
- **DML (Data Manipulation Language):** INSERT, UPDATE, DELETE, MERGE.
- **DQL (Data Query Language):** SELECT.
- **DCL (Data Control Language):** GRANT, REVOKE.
- **TCL (Transaction Control Language):** BEGIN, COMMIT, ROLLBACK, SAVEPOINT.

2.2 Creating, Inserting, Deleting, and Updating

2.2.1 CREATE TABLE

Listing 1: CREATE TABLE Syntax

```
CREATE TABLE <table_name> (
  <attr_1> <type_1> [<column_constraint_1>],
  ...
```

```

<attr_n> <type_n> [<column_constraint_n>],
[<table_constraint_1>],
...
[<table_constraint_m>]
);

```

- SQL is case-insensitive.
- **Semantics:** Defines a relation schema $\langle \text{table_name} \rangle (\langle \text{attr}_1 \rangle : \langle \text{type}_1 \rangle, \dots)$.

2.2.2 INSERT INTO

Listing 2: INSERT INTO Syntax

```

INSERT INTO <table_name> [(<attr_1>, ..., <attr_n>)]
VALUES
(<val_1_1>, ..., <val_n_1>),
...
(<val_2_m>, ..., <val_n_m>);

```

- Either all inserted or none are inserted.
- Missing trailing values are replaced with NULL (if allowed) or default values (if specified using DEFAULT).

2.2.3 DELETE FROM

Listing 3: DELETE FROM Syntax

```

DELETE FROM <table_name>
[WHERE <condition>];

```

- **Principle of Acceptance:** The operation is performed if the $\langle \text{condition} \rangle$ evaluates to True.
- If $\langle \text{condition} \rangle$ is unspecified, all rows are deleted.

2.2.4 UPDATE

Listing 4: UPDATE Syntax

```

UPDATE <table_name>
SET <column_1> = <expr_1>
    [, <column_2> = <expr_2>]
    [...]
[WHERE <condition>];

```

- Also uses the principle of acceptance for the **WHERE** clause.

- The operation may depend on the order of rows when the update happens, especially related to other constraints.

2.3 Three-Valued Logic in SQL

- SQL uses three truth values: **True**, **False**, and **NULL** (Unknown).
- **Mental Model**: NULL values are ALL possible values at the same time.

2.3.1 Logical Operations

- **Conjunction (AND)**: If either operand is **False**, the result is **False**. If both are **NULL**, the result is **NULL**. If one is **True** and the other **NULL**, the result is **NULL**. Only **True AND True** is **True**.
- **Disjunction (OR)**: If either operand is **True**, the result is **True**. Only **False OR False** is **False**. All other combinations involving **NULL** are **NULL**.

2.3.2 Relational and Arithmetic Operations

- Any relational comparison or arithmetic operation with **NULL** produces a **NULL** result.
- E.g., $\text{NULL} < 2024 \rightarrow \text{NULL}$; $\text{NULL} + 0 \rightarrow \text{NULL}$; $\text{NULL} = \text{NULL} \rightarrow \text{NULL}$.

2.3.3 Handling NULL as Values

- $V1 \text{ IS NULL} / V1 \text{ IS NOT NULL}$
- $V1 \text{ IS DISTINCT FROM } V2 / V1 \text{ IS NOT DISTINCT FROM } V2$ (treats two **NULL** values as not distinct/equal).

2.4 Integrity Constraints in SQL

2.4.1 Preliminary

- **Principle of Rejection**: We reject the insertion if the constraint condition evaluates to **False**. If it evaluates to **NULL** or **True**, the insertion is performed.
- Constraints can be defined at the **Column Constraint** level (applies to a single column) or **Table Constraint** level (applies to one or more columns).

2.4.2 Specific Constraints

- **NOT NULL** (*Column Constraint*): Rejects insertion if the value in the specified column is **NULL**. Condition: $A \text{ IS NOT NULL}$ must not be **False**.

- **UNIQUE** (*Column or Table Constraint*): Rejects insertion if there are other rows where the values are equal. NULL values are generally excluded from uniqueness checks (i.e., $\text{NULL} \neq \text{NULL} \rightarrow \text{NULL}$, not **False**).
- **PRIMARY KEY** (*Column or Table Constraint*): Equivalent to **UNIQUE** and **NOT NULL** combined. For a composite primary key, every individual column must be **NOT NULL**.
- **FOREIGN KEY** (*Table Constraint*): Enforces referential integrity.

2.4.3 Foreign Key Actions

When a row in the referenced table is updated or deleted, the action defines the behavior:

- **NO ACTION** (Default): Reject delete/update if it violates the constraint.
- **RESTRICT**: Similar to **NO ACTION**, but cannot be deferred.
- **CASCADE**: Propagates delete/update to the referencing tuples.
- **SET DEFAULT**: Updates the foreign key of the referencing tuples to a default value. The default value must exist as a primary key in the referenced table.
- **SET NULL**: Updates the foreign key of the referencing tuples to **NULL**. The corresponding column must allow **NULL** values.

2.4.4 General Check Constraint

- **CHECK** (*Column or Table Constraint*): Allows specification of an arbitrary Boolean expression that must not evaluate to **False**.
- **Assertion** (**CREATE ASSERTION**): Used for arbitrary constraints across multiple tables/rows. Most RDBMS do **NOT** support it; alternatives are triggers.

2.5 Modifying Database (**ALTER** and **DROP**)

2.5.1 ALTER TABLE

Listing 5: ALTER TABLE Syntax

```
ALTER TABLE <table_name>
[ALTER / ADD / DROP] [COLUMN / CONSTRAINT] <name> <changes>;
```

- Used for adding/dropping columns or constraints, and changing column specifications (data type, default value).

2.5.2 DROP TABLE

Listing 6: DROP TABLE Syntax

```
DROP TABLE [IF EXISTS] <table_name>[, ...] [CASCADE];
```

- **IF EXISTS:** No error if the table does not exist.
- **CASCADE:** Also deletes referencing tables.

2.6 Deferrable Constraints

2.6.1 Motivation

- **Default behavior:** Constraints are checked immediately at the end of every SQL statement. A violation causes the statement/transaction to be rolled back.
- **Relaxed Constraint:** Check only at the end of a transaction (available for **UNIQUE**, **PRIMARY KEY**, **FOREIGN KEY**). This allows for temporarily inconsistent intermediate states, useful for circular foreign key references or batch operations.

2.6.2 Deferment

- **NOT DEFERRABLE:** Constraint is checked immediately (default).
- **DEFERRABLE INITIALLY DEFERRED:** Constraint check is delayed until the transaction commits.
- **DEFERRABLE INITIALLY IMMEDIATE:** Constraint is checked immediately, but can be explicitly deferred for the current transaction using **SET CONSTRAINTS constraint_name DEFERRED;**.

3 L03: Entity-Relationship (ER) Model

3.1 Database Design Process

1. **Requirement Analysis:** Identification and collection of user needs (data, application, performance).
2. **Conceptual Database Design:** Capturing requirements using a conceptual model (e.g., ER Model).
3. **Logical Database Design:** Mapping the conceptual model to a logical schema (e.g., ER Model $\rightarrow$ Relational Schema).
4. **Schema Refinement:** Refining the relational schema (normalization) to check for and remove redundancies and anomalies.

5. **Physical Database Design:** Implementing the database based on the final schema with consideration of performance requirements.
6. **Security Design:** Identification of users, user groups, and their access permissions.

3.2 ER Model Concepts and Elements

3.2.1 ER Model

- The most common model for conceptual database design, developed by Peter Chen in 1976.
- Visualized using **ER Diagrams**.

3.2.2 Core Concepts

- **Entities** (Nouns, Rectangles): Representation of distinguishable real-world objects (e.g., Users, Flights).
- **Entity Set** (Rectangle): A collection of entities of the same type. Names are typically plural nouns, not proper nouns.
- **Relationships** (Verbs, Diamonds): An association among one or more entities (e.g., Users **makes** Bookings).
- **Relationship Set** (Diamond): A collection of relationships of the same type. May have its own attributes.
- **Attributes** (Ovals): Specific information describing an entity or a relationship.

3.2.3 Attribute Kinds

- **Key Attributes** (Underlined): Uniquely identifies each entity (e.g., uid).
- **Composite Attributes** (Composed of other ovals, 1-level only): Composed of multiple other attributes (e.g., **address** → **street**, **block**, **zip**).
- **Multi-Valued Attributes** (Double-lined): One or more values for a given entity (e.g., **phone**).
- **Derived Attributes** (Dashed line): Derived from other attributes (e.g., **age** from **dob**). Should not appear in the relational schema.

3.2.4 Relationship Degrees

- **Degree:** The number of entity sets involved in the relationship set.
- **Binary** ($n = 2$), **Ternary** ($n = 3$), etc. Binary relationships are the most common.

3.3 Relationship Constraints (Look Here Notation)

3.3.1 Cardinality Constraints (Upper Bound)

Describes an upper bound (1 or ∞) to the number of times an entity can participate in a relationship.

- **Line (Default):** Upper Limit = ∞ (Many).
- **Arrow ($\rightarrow$):** Upper Limit = 1 (One).
- **Many-to-Many ($\infty : \infty$):** Line on both sides.
- **Many-to-One ($\infty : 1$):** Line on the "many" side, arrow on the "one" side (or vice versa).
- **One-to-One ($1 : 1$):** Arrow on both sides.

3.3.2 Participation Constraints (Lower Bound)

Describes a lower bound (0 or 1) to the number of times an entity can participate in a relationship.

- **Single Line (Default):** Lower Limit = 0 (**Partial Participation**).
- **Double Line (=):** Lower Limit = 1 (**Total Participation**).
- **Key + Total Participation:** An entity participates in *exactly 1* instance of the relationship (Arrow and Double Line on the connection).

3.3.3 Dependency Constraints (Weak Entity Sets)

- **Weak Entity Set** (Double-lined Rectangle): An entity set that does *not* have its own key. Its key is called a **partial key** (underlined with a dashed line).
- **Identifying Relationship Set** (Double-lined Diamond): The relationship connecting the weak entity set to its **owning entity set**.
- **Requirements:** The weak entity set must satisfy a Key + Total Participation constraint with respect to the identifying relationship set. Its existence is dependent on the existence of its owner.

3.4 Relational Mapping (ER Diagram to Schema)

3.4.1 Entity Set Mapping

- **Basic Mapping:** Entity Set $\rightarrow$ Table. Attributes $\rightarrow$ Table Columns. Key Attribute $\rightarrow$ Primary Key.
- **Derived Attribute:** Should not appear (is computed).

- **Composite Attribute:** Converted into a series of single-valued attributes (e.g., `address` $\rightarrow$ `street`, `block`, `zip`).
- **Multi-Valued Attribute:** Requires a **separate table** with a foreign key constraint linking back to the owning entity's primary key (unless multiplicity is fixed).

3.4.2 Relationship Set Mapping (Binary)

1. **Many-to-Many** ($\infty : \infty$): Create a **separate table** for the relationship set. This table includes:
 - The primary key attributes of all participating entity sets (as foreign keys).
 - All attributes of the relationship set itself.
 - The combination of the primary keys of the participating entities forms the primary key of the new table.
2. **Many-to-One** ($\infty : 1$): Combine tables (or keep separate tables).
 - **Separate Tables (Strategy #1):** Create a separate table for the relationship. The primary key of the "many" side acts as the primary key of the relationship table. The key of the "one" side is a foreign key.
 - **Combined Tables (Strategy #2 - Preferred):** Add the primary key of the "one" side entity set as a foreign key attribute to the table representing the "many" side entity set.
3. **One-to-One** ($1 : 1$): Foreign key in one table referring to the other, potentially combined with **UNIQUE** constraints. Can be combined into one table with a composite key or split into three tables with a relationship table in the middle.
4. **Key + Total Participation** ($1 : 1$ or $\infty : 1$ with total participation on the 1 side): Typically implemented using a **combined table** approach, enforcing the foreign key using **NOT NULL** on the foreign key attribute in the referencing table.
5. **Weak Entity Set:** Always translated to a **combined table** with its owning entity set via the identifying relationship. The resulting table includes:
 - The primary key of the owner entity set (as a foreign key, must be **NOT NULL**).
 - The partial key of the weak entity set.
 - The combination of the owner's primary key and the partial key forms the composite primary key of the new table.
 - Use **ON DELETE CASCADE** to enforce the existence dependency.

3.5 Extended Notations

3.5.1 ISA Hierarchies (Generalization/Specialization)

- Represents the "is a" relationship (superclass/subclass).
- Subclasses (specialized entity sets) inherit all attributes and keys from the superclass (generalized entity set). Keys are typically not shown in the subclass.
- **Overlap Constraints:** Can a superclass entity belong to multiple subclasses?
- **Covering Constraints:** Must a superclass entity belong to at least 1 subclass?
- **Relational Mapping (General Strategy):** One relation per generalized and specialized entity sets. The primary key of the subclass table is a foreign key referencing the superclass table with `ON DELETE CASCADE`. This works well when `overlap=TRUE` and `covering=FALSE`.

3.5.2 Aggregation

- Allows a **relationship set** to be treated as a **higher-level entity set** (i.e., participate in other relationships).
- Represented as a diamond inside a rectangle. The connecting line is to the rectangle, not the diamond.
- **Relational Mapping:** Create a new table for the aggregate relationship. The key attributes of the aggregate entity form the primary key of the new table. Foreign keys are created to the tables corresponding to the entities involved in the aggregate relationship.

4 L04: SQL (Part 1)

4.1 Basic SQL Queries

4.1.1 SELECT-FROM-WHERE Structure

Listing 7: Basic SQL Query Structure

```
SELECT [ DISTINCT ] <target-list>  -- SELECT clause
FROM <relation-list>                -- FROM clause
WHERE <conditions>;                 -- WHERE clause
```

- **SELECT** $\langle$ target-list $\rangle$: List of attributes/columns for the output result. Use `*` for all attributes.
- **FROM** $\langle$ relation-list $\rangle$: List of relations relevant to the query.

- **WHERE** $\langle \text{conditions} \rangle$: Conditions that specify a valid row/tuple (using Principle of Acceptance, i.e., condition must be **True**).
- **DISTINCT**: Optional keyword to eliminate duplicate rows (SQL generally uses multi-set/bag semantics by default).
- Output is typically **Order-Independent** unless **ORDER BY** is used.

4.1.2 Conceptual Evaluation Steps

1. Compute the Cartesian product (join) of all relations in the **FROM** clause.
2. Keep only rows where the $\langle \text{condition} \rangle$ in the **WHERE** clause evaluates to **True**.
3. Keep only attributes specified in the $\langle \text{target-list} \rangle$.
4. Remove duplicate rows if **DISTINCT** is specified.

4.1.3 SELECT Clause Details

- **Expression**: Can include mathematical, string, or date/time operations (e.g., `price * 1.34`).
- **Renaming**: Use **AS** to rename columns (e.g., `price * 1.34 AS sgd`).
- **DISTINCT NULL**: The **DISTINCT** keyword removes duplicate **NULL** values by using **IS DISTINCT FROM** check.

4.1.4 WHERE Clause Details

- **NULL Values**: Use `x IS NULL` or `x IS NOT NULL`. Using `x = NULL` or `x <> NULL` is almost always wrong, as they evaluate to **NULL** (and thus fail the Principle of Acceptance unless used within other constructs like a subquery).
- **Pattern Matching (Basic)**: `column [NOT] LIKE pattern`.
 - `_`: Matches any single character.
 - `%`: Matches any sequence of 0 or more characters.

4.2 Set Operations

- Two relations R and S are **union-compatible** if they have the same number of attributes and corresponding attributes have the same or compatible domains.
- **UNION** ($\cup$), **INTERSECT** ($\cap$), **EXCEPT** (set difference $-$): Automatically remove duplicates.
- **UNION ALL**, **INTERSECT ALL**, **EXCEPT ALL**: Keep duplicate rows.

- Column names in the result follow the left relation.

4.3 Join Operations

4.3.1 Cartesian Join (Cross Product)

```
SELECT * FROM R1, R2;
```

- Pairs every row in R_1 with every row in R_2 .
- Result size: $|R_1| \times |R_2|$.

4.3.2 Inner Join

- Equivalent to a Cartesian product followed by a selection (using a `WHERE` clause condition).
- Syntax (explicit): `FROM R1 JOIN R2 ON condition`
- Syntax (implicit): `FROM R1, R2 WHERE condition`

4.3.3 Natural Join ($\bowtie$)

```
SELECT * FROM R1 NATURAL JOIN R2;
```

- Performs a join over **all common attributes** between R_1 and R_2 , equating the values for those common attributes.
- The output relation contains the common attributes only once.
- If no common attributes, it defaults to a Cartesian join.

4.3.4 Outer Join

- **Motivation:** Inner joins remove "dangling tuples" (tuples that do not satisfy the join condition). Outer joins retain them, padding missing values with `NULL`.
- **LEFT [OUTER] JOIN:** Retains all tuples from the left relation (R_1).
- **RIGHT [OUTER] JOIN:** Retains all tuples from the right relation (R_2).
- **FULL [OUTER] JOIN:** Retains all tuples from both R_1 and R_2 .
- Finding dangling tuples (e.g., those from R_1 that did not match R_2 in a left join):
Use a left join and filter with `R2.attribute IS NULL`.

4.4 Composing Queries with Subqueries

4.4.1 Subqueries

- Queries nested within another query (in **SELECT**, **FROM**, or **WHERE** clauses).
- **Correlated Subqueries**: The inner query uses values from the outer query. Typically, the inner query is executed once per row of the outer query.
- **Uncorrelated Subqueries**: The inner query executes independently.

4.4.2 Scalar Subquery

- A subquery that returns **at most a single value** (1 row, 1 column).
- Can be used as a value in the **SELECT** or **WHERE** clauses.
- If the result is 0 rows, it is treated as **NULL**.

4.4.3 Set Membership ([NOT] IN)

... WHERE <expr> [NOT] IN <subquery>;

- Subquery must return **exactly one column**.
- **IN**: Returns **True** if <expr> matches any row in <subquery>. ($\exists \text{row} \in \langle \text{subquery} \rangle : \text{row} = \langle \text{expr} \rangle$)
- **NOT IN**: Returns **True** if <expr> matches none of the rows. ($\forall \text{row} \in \langle \text{subquery} \rangle : \text{row} \neq \langle \text{expr} \rangle$)
- **Empty Subquery**: **IN** returns **False**; **NOT IN** returns **True**.
- **IN** is generally equivalent to an inner join; **NOT IN** is equivalent to an outer join detecting dangling tuples.

4.4.4 Existence ([NOT] EXISTS)

... WHERE [NOT] EXISTS <subquery>;

- Subquery can return any number of columns. Only checks if the result set is non-empty.
- **EXISTS**: Returns **True** if the <subquery> result set is non-empty.
- **NOT EXISTS**: Returns **True** if the <subquery> result set is empty.

4.4.5 Quantification (ANY / ALL)

... WHERE <expr> <op> { ANY | ALL } <subquery>;

- Subquery must return **exactly one column**.
- **ANY**: Returns **True** if the comparison is **True** for **at least one** row in the subquery. (<expr> IN <subquery>)
- **ALL**: Returns **True** if the comparison is **True** for **all** rows in the subquery.

4.4.6 Scoping Rules

- Attributes are resolved from the inner query scope outwards (inner to outer nesting).
- To avoid ambiguity, use **table aliases** (R1.attribute) for all attributes.

4.5 Ordering

4.5.1 ORDER BY

... ORDER BY <attr> [ASC | DESC] [, <attr> [ASC | DESC] ...];

- Sorts the result set by one or more attributes. **ASC** (ascending) is the default; **DESC** is descending.
- If **DISTINCT** is used, the sorting attributes must appear in the **SELECT** clause.

4.5.2 LIMIT & OFFSET

- **LIMIT k**: Returns the first k rows.
- **OFFSET i**: Skips the first i rows before returning the result.
- Typically combined with **ORDER BY** to return a specific segment of a sorted result (e.g., the 5 most expensive items).

5 L05: SQL (Part 2)

5.1 Common SQL Constructs

5.1.1 Aggregation Functions

- Compute a single value from a set of tuples.
- **MIN()**, **MAX()**, **SUM()**, **COUNT()**, **AVG()**.

- **Interpretation:** Aggregation functions ignore NULL values in their calculations (e.g., COUNT(A) counts non-NULL values of A). An exception is COUNT(*) which counts all rows, including those with NULL values.
- **Empty Set Semantics:** If the relation is empty or all values are NULL, MIN/MAX/SUM/AVG return NULL. COUNT returns 0.
- **DISTINCT:** Used inside the function to aggregate over unique non-NULL values (e.g., AVG(DISTINCT A)).

5.1.2 Grouping (GROUP BY)

```
... GROUP BY attr1, attr2, ...
```

- Logically partitions the relation into groups based on values of the specified attributes.
- Aggregation functions then apply to each group individually, resulting in one output tuple per group.
- **Restriction:** Any attribute in the SELECT clause must either appear in the GROUP BY clause or be an input to an aggregate function.

5.1.3 Filtering Groups (HAVING)

```
... GROUP BY ...  
HAVING <condition>
```

- Filters the groups created by GROUP BY.
- Conditions typically involve aggregate functions.
- **Restriction:** Similar to the SELECT clause, attributes in the HAVING clause must either appear in GROUP BY or be inputs to aggregate functions.
- **Conceptual Evaluation:** WHERE filters tuples *before* grouping; HAVING filters groups *after* grouping.

5.1.4 Conditional Expressions

- **CASE:** Returns a result based on specified conditions or value matches. Only one result is returned (the first match). ELSE is optional (returns NULL if omitted).
- **COALESCE(val_1, val_2, ...):** Returns the **first non-NULL value** in the list of input arguments. Returns NULL if all are NULL.
- **NULLIF(value_1, value_2):** Returns NULL if value_1 = value_2; otherwise, returns value_1. Useful for safely ignoring specific values in aggregation (e.g., AVG(NULLIF(price, 1)) to ignore prices of 1).

5.2 Structuring Queries

5.2.1 Table Expressions (WITH)

Listing 8: Common Table Expression (CTE) Syntax

```
WITH
  CTE_1 AS ( Q_1 ),
  CTE_2 AS ( Q_2 ),
  ...
  CTE_n AS ( Q_n )
Q_0
```

- **Common Table Expressions (CTEs)** define temporary, named result sets that exist only for the duration of the main query Q_0 .
- CTEs can reference other previously defined CTEs (i.e., CTE_i can reference CTE_j where $j < i$).
- Used to improve readability, debugging, and maintenance of complex queries.
- Queries wrapped in parentheses in the **FROM** clause also act as a temporary table expression.

5.2.2 Views (CREATE VIEW)

Listing 9: CREATE VIEW Syntax

```
CREATE VIEW <name> (<attr_1>, ..., <attr_n>) AS
  <query>
;
```

- A **permanently named query** (virtual table). Conceptually, no actual table is created; the result is computed when the view is accessed.
- Used to represent **External Schema**, simplify complex queries, and manage security (hiding data).
- **Updatable VIEW**: Restricted for INSERT, UPDATE, and/or DELETE. Generally, a view must be built upon a single base table without complex logic (e.g., no DISTINCT, GROUP BY, joins, or aggregation).

5.3 Extended Concepts

5.3.1 Universal Quantification ($\forall$) in SQL

- SQL directly supports only **existential quantification** (EXISTS or ANY).
- Universal quantification (**ALL**) problems ($\forall x : P(x)$) can be solved by transforming the logic:

1. **Double Negation (Set Containment):** $\forall x : P(x) \Leftrightarrow \neg \exists x : \neg P(x)$
 - Example: "Sells ALL pizzas liked by Homer" $\Leftrightarrow$ "NOT EXISTS (pizza x liked by Homer AND NOT sold by restaurant R)"
2. **Cardinality Check (Set Equality):** If S is the set of items to check, and R is the set of items a group has, then R contains all of S if $|R \cup S| = |R|$.

5.3.2 Recursive Queries

- Used to query hierarchical or graph-like data (e.g., finding all reachable MRT stations).
- Implemented using **Recursive CTEs**.

Listing 10: Recursive CTE Syntax

```
WITH RECURSIVE CTE_name AS (
  Q_1 -- Non-recursive base case
  UNION [ ALL ]
  Q_2 ( CTE_name ) -- Recursive step referencing CTE_name
)
Q_0 ( CTE_name ) -- Main query
```

- Q_1 (base case) defines the starting points.
- Q_2 (recursive case) repeatedly joins the current state of CTE_name with the base relation until a fixed-point (no new rows) is reached.

6 L06: Relational Algebra (RA)

6.1 Introduction to Relational Algebra

6.1.1 Definition

- A mathematical system consisting of **Operands** (relations) and **Operators** (transformations from one or more input relations into one output relation).

6.1.2 Why Relational Algebra?

- Provides a formal basis for database query languages like SQL.
- Supports **Query Optimization**: Expressions can be algebraically rewritten (using equivalence laws) for faster evaluation (lower cost).

6.1.3 Closure Property

- The set of all relations is **closed** under the set of relational algebra operators.

- This means the output of one RA operator is always a relation and can thus be used as the input to another RA operator, allowing for the chaining of operations.

6.2 Relational Algebra Operators

6.2.1 Unary Operators

1. **Selection (σ):** $\sigma_C(R)$
 - Selects all tuples (rows) from relation R that satisfy the condition C .
 - Maps to the **WHERE** clause in SQL.
 - **Property:** Result has the same schema as R .
2. **Projection (π):** $\pi_L(R)$
 - Keeps only the columns specified in the ordered list L and in that same order.
 - Automatically removes duplicate rows because relations are defined as a set of tuples.
 - Maps to the column list in the **SELECT** clause (implicitly equivalent to **SELECT DISTINCT** for sets).
 - **Property:** The order of attributes in L matters ($\pi_{A,B}(R) \neq \pi_{B,A}(R)$).
3. **Renaming (ρ):** $\rho_{B \leftarrow A}(R)$
 - Renames the attribute A in relation R to B .
 - Can also rename the entire relation: $\rho_{S(B \leftarrow A)}(R)$.
 - Maps to the **AS** keyword in the SQL **SELECT** clause.
 - **Property:** The number of rows is unchanged.

6.2.2 Binary Operators

1. **Set Operators (Requires Union-Compatibility):**
 - **Union ($\cup$):** $R \cup S$ (tuples in R OR S).
 - **Intersection ($\cap$):** $R \cap S$ (tuples in R AND S).
 - **Difference (Minus, $-$):** $R - S$ (tuples in R BUT NOT S). Asymmetric operation.
2. **Product (Cross Product, $\times$):** $R \times S$

- Combines all pairs of tuples from R and S . Requires R and S to have no common attributes.
- The resulting schema is a concatenation of all attributes in R and S .

3. Joins (Inner Joins):

- **Theta-Join** ($\bowtie_\theta$): $R \bowtie_\theta S = \sigma_\theta(R \times S)$. Cross product followed by selection on condition θ .
- **Equi-Join** ($\bowtie_=\$): A special theta-join where θ only uses equality operators.
- **Natural Join** ($\bowtie$): $R \bowtie S$. A special equi-join performed implicitly over all common attributes. The result projects out duplicate columns.

4. Joins (Outer Joins): Based on an inner join $R \bowtie_\theta S$.

- **Left Outer Join** ($\leftarrow_\theta$): $R \leftarrow_\theta S = (R \bowtie_\theta S) \cup (\text{dangle}(R \bowtie_\theta S) \times \{\text{null}(S)\})$. Keeps unmatched tuples from R .
- **Right Outer Join** ($\rightarrow_\theta$): $R \rightarrow_\theta S = (R \bowtie_\theta S) \cup (\{\text{null}(R)\} \times \text{dangle}(S \bowtie_\theta R))$. Keeps unmatched tuples from S .
- **Full Outer Join** ($\leftrightarrow_\theta$): $R \leftrightarrow_\theta S = (R \leftarrow_\theta S) \cup \dots \cup \dots$. Keeps unmatched tuples from both R and S .

6.3 Complex Expressions and Equivalence

6.3.1 Expression Tree

- Complex RA expressions can be visualized as an operator tree, where leaves are relations and internal nodes are operators.

6.3.2 Equivalence and Isomorphism

- **Equivalent** ($\equiv$): Two expressions Q_1 and Q_2 produce the same result set with the **same column order**.
- **Isomorphic** ($\cong$): Two expressions produce the same result set, possibly with **different column order**.

6.3.3 Equivalence Properties (Optimization Rules)

- **Selection Commutativity**: $\sigma_{\theta_1}(\sigma_{\theta_2}(R)) \equiv \sigma_{\theta_2}(\sigma_{\theta_1}(R)) \equiv \sigma_{\theta_1 \wedge \theta_2}(R)$.
- **Projection Reduction**: $\pi_{L_1}(\pi_{L_2}(R)) \equiv \pi_{L_1}(R)$ only if $L_1 \subseteq L_2$.
- **Join Associativity**: $R \times (S \times T) \equiv (R \times S) \times T$. $R \bowtie (S \bowtie T) \cong (R \bowtie S) \bowtie T$.

- **Pushing Selection Down (Query Rewriting):** $\sigma_\theta(R \times S) \equiv \sigma_\theta(R) \times S$ only if θ only involves attributes in R . This reduces the size of the intermediate relation early.

7 L07: Programming with SQL: Functions, Procedures, and Cursors

7.1 Accessing Database from Host Languages

7.1.1 Mixing Host Language with SQL (Statement-Level Interface - SLI)

- Write a program mixing a host language (e.g., C, Java, Python) with SQL statements.
- A preprocessor converts the mixed code into pure host language code.
- SQL statements are prefixed with `EXEC SQL`. Shared host language variables used in SQL must be preceded by a colon (`:var`).
- **Static SQL:** SQL statement is fixed beforehand (e.g., `EXEC SQL INSERT INTO ... VALUES(:var)`).
- **Dynamic SQL:** SQL statement is constructed as a string variable and compiled/executed at runtime (e.g., `EXEC SQL EXECUTE IMMEDIATE :stmt`).
- **Prepared Statements:** A dynamic SQL variant where the statement is parsed once and executed multiple times with different parameters using placeholders (?).

7.1.2 Call-Level Interface (CLI)

- Application is written entirely in the host language. SQL statements are executed via function calls to APIs (e.g., JDBC for Java, ODBC for C/C++).

7.1.3 SQL Injection Attacks

- Occurs when user-provided input is concatenated directly into a dynamically constructed SQL string (`EXECUTE IMMEDIATE`).
- A malicious user can inject SQL commands (e.g., `a'; DROP TABLE T; SELECT 'a)` to alter the database schema or data.
- **Solutions:** Use functions/procedures (where the input is treated as a string constant) or prepared statements (where user input is parameter, not part of the compiled query structure).

7.2 Functions and Procedures in SQL (PL/pgSQL)

7.2.1 Motivation

- Write functions/procedures directly in the database for code reuse, ease of maintenance, performance, and security.
- PostgreSQL uses PL/pgSQL (Procedural Language/PostgreSQL SQL).

7.2.2 SQL Functions (CREATE OR REPLACE FUNCTION)

- **Returns:** Functions must return a value. Options include:
 - * A single scalar value (`RETURNS char(1)`).
 - * An existing tuple (`RETURNS Scores`) or a set of existing tuples (`RETURNS SETOF Scores`).
 - * A new tuple (`RETURNS RECORD`) using OUT parameters.
 - * A new set of tuples (`RETURNS SETOF RECORD` or `RETURNS TABLE(col TYPE, ...)`).
- Can be called within a `SELECT` statement.

7.2.3 SQL Procedures (CREATE OR REPLACE PROCEDURE)

- Used when no return value/tuple is needed (e.g., performing updates).
- Executed using `CALL procedure_name(...)`.

7.2.4 Control Structures

PL/pgSQL supports procedural programming elements:

- `DECLARE` block for variables.
- `IF ...THEN ...END IF`.
- `LOOP ...EXIT WHEN ...END LOOP`.
- `WHILE ...LOOP ...END LOOP`.

7.2.5 Cursors

- Enables row-by-row processing of a result set returned by a `SELECT` statement.
- **Workflow:** `DECLARE` → `OPEN` → `LOOP/FETCH` (moves to the next row) → `EXIT WHEN NOT FOUND` → `CLOSE`.
- Can fetch rows into variables or `RECORD` types.

- **Movement:** FETCH NEXT (default), FETCH PRIOR, FETCH FIRST/LAST, FETCH ABSOLUTE *n*.

8 L08: Programming with SQL: Triggers and Normal Forms (Part 1)

8.1 Triggers

8.1.1 Motivation

- A mechanism to automatically execute a special function (**trigger function**) whenever a specified database event (INSERT, UPDATE, DELETE) occurs on a specified table or view.

8.1.2 Trigger Definition (CREATE TRIGGER)

Listing 11: CREATE TRIGGER Syntax

```
CREATE TRIGGER <name>
{BEFORE | AFTER | INSTEAD OF} {INSERT | DELETE | UPDATE [OF col] | TRUNCATE}
[FOR EACH {ROW | STATEMENT}]
[WHEN (condition)]
EXECUTE FUNCTION <trigger_function>();
```

8.1.3 Trigger Function (CREATE OR REPLACE FUNCTION ...RETURNS TRIGGER)

- Must return TRIGGER (not RECORD) to access special contextual variables.
- **Contextual Variables:**
 - * TG_OP: The operation that activated the trigger ('INSERT', 'UPDATE', 'DELETE', etc.).
 - * TG_TABLE_NAME: The name of the table.
 - * NEW: The new row for INSERT and UPDATE operations.
 - * OLD: The old row for UPDATE and DELETE operations.

8.1.4 Trigger Timing

- **BEFORE:** Executes *before* the operation. Can modify the NEW row or skip the operation.
- **AFTER:** Executes *after* the operation. Cannot modify the row or skip the operation.

- **INSTEAD OF**: Only allowed on **views**. Executes *instead* of the operation (e.g., update the underlying base table instead of the view).

8.1.5 Return Values

- **BEFORE/INSTEAD OF**:
 - * Return **NULL**: Skips the rest of the operation for the current row (in row-level triggers).
 - * Return a non-**NULL** tuple (e.g., **RETURN NEW**): The returned tuple is used for the operation (e.g., the row to be inserted/updated).
- **AFTER**: Return value is ignored.
- **Statement-Level Trigger Return**: Ignored. To omit the subsequent operation, raise an exception (**RAISE EXCEPTION**).

8.1.6 Trigger Levels

- **FOR EACH ROW (Row-level)**: Trigger function executes for every tuple encountered.
- **FOR EACH STATEMENT (Statement-level)**: Trigger function executes only once per SQL statement, regardless of how many rows it affects.
- **INSTEAD OF** is only allowed on row-level.
- Deferred triggers only work with **AFTER** and **FOR EACH ROW**.

8.1.7 Trigger Condition (WHEN)

- Optional **WHEN (condition)** clause that specifies when to execute the trigger function (only for row-level triggers).
- Reduces the overhead of calling the function.
- Restrictions: No **SELECT**, no **OLD** for **INSERT**, no **NEW** for **DELETE**.

8.1.8 Deferred Trigger (CONSTRAINT TRIGGER)

- **CREATE CONSTRAINT TRIGGER** combined with **DEFERRABLE [INITIALLY DEFERRED | IMMEDIATE]** allows the trigger check to be deferred until the end of the transaction (**COMMIT**).
- This is crucial for transactions that temporarily violate constraints (e.g., fund transfers).
- **DEFERRABLE** triggers must be **AFTER** and **FOR EACH ROW**.

8.2 Normal Forms (NF) and Functional Dependencies (FD)

8.2.1 Motivation

- Poorly designed relational schemas lead to **data redundancy** and **anomalies**.
- **Update Anomalies**: Accidentally updating one redundant copy but leaving others unchanged.
- **Deletion Anomalies**: Deleting a row causes unintended data loss (e.g., losing information about an address when the last phone number row is deleted).
- **Insertion Anomalies**: Cannot insert a new fact (e.g., an address for a person) until an associated fact exists (e.g., a phone number is provided).
- **Normalization**: Decomposing (splitting) tables to eliminate redundancy and anomalies.

8.2.2 Functional Dependency (FD)

- **Definition**: $A_1A_2 \dots A_m \rightarrow B_1B_2 \dots B_n$ (Read as: $A_1 \dots A_m$ *decides* $B_1 \dots B_n$).
- Means: Whenever two tuples have the same values on $A_1, \dots, A_m$, they must also have the same values on $B_1, \dots, B_n$.
- FDs come from common sense and application requirements.

8.2.3 FD Reasoning (Armstrong's Axioms)

- **Axiom of Reflexivity**: $A_1 \dots A_n \rightarrow A_i \dots A_j$ (A set of attributes implies any subset of itself).
- **Axiom of Augmentation**: If $A \rightarrow B$, then $AC \rightarrow BC$ for any attribute set C .
- **Axiom of Transitivity**: If $A \rightarrow B$ and $B \rightarrow C$, then $A \rightarrow C$.
- **Derived Rules (Union and Decomposition)**:
 - * **Decomposition**: If $A \rightarrow BC$, then $A \rightarrow B$ and $A \rightarrow C$.
 - * **Union**: If $A \rightarrow B$ and $A \rightarrow C$, then $A \rightarrow BC$.

8.2.4 Closure of an Attribute Set (X^+)

- **Definition**: Given a set of attributes X , the closure X^+ is the set of all attributes that can be decided by X (directly or indirectly) using the given FDs.
- **Computation**: Start with X , and repeatedly apply the FDs until no new attributes can be added.

- **FD Proof:** $X \rightarrow Y$ holds if and only if $Y \subseteq X^+$.

8.2.5 Keys and Superkeys via FDs

- **Superkey** of a table R : A set of attributes K such that $K^+ = \text{Attr}(R)$ (all attributes in R).
- **Key** of a table R : A superkey that is minimal (no proper subset is also a superkey).
- **Algorithm for finding keys:** Compute X^+ for all subsets X . Any X that yields $X^+ = \text{Attr}(R)$ is a superkey. The minimal ones are the keys.
- **Prime Attribute:** An attribute that appears in at least one key.
- **Non-Prime Attribute:** An attribute that appears in no key.

9 L09: Normal Forms (Part 2)

9.1 BCNF (Boyce-Codd Normal Form)

9.1.1 Non-trivial and Decomposed FDs

- **Decomposed FD:** An FD whose right hand side has only one attribute (e.g., $A \rightarrow C$).
- **Non-trivial and Decomposed FD:** A decomposed FD $X \rightarrow A$ where A does not appear in X ($A \notin X$).
- NF checks typically focus on this simplified set of FDs.

9.1.2 BCNF Definition

- A table R is in BCNF if for every non-trivial and decomposed FD $X \rightarrow A$ that holds on R , the left hand side X must be a **superkey** of R .
- **Intuition:** All attributes must depend only on superkeys; any dependency on a non-superkey is prohibited, eliminating redundancy.

9.1.3 Simplified BCNF Check (Using Closure)

- A table R is **NOT** in BCNF if and only if there exists a closure X^+ (for some attribute set X) that satisfies the **”more but not all”** property:
 1. The closure X^+ contains more attributes than X does ($X \subset X^+$).
 2. The closure X^+ does not contain all attributes in the table ($X^+ \subset \text{Attr}(R)$).

- If such a closure X^+ exists, it means $X \rightarrow A$ holds for some $A \notin X$, and X is not a superkey, thus violating BCNF.

9.1.4 BCNF Decomposition

- **Goal:** Decompose a non-BCNF table into smaller tables that are all in BCNF.
- **Algorithm (Binary Split):**
 1. Find a minimal set X such that X^+ violates BCNF (i.e., X^+ satisfies the "more but not all" property).
 2. Decompose R into two tables:
 - * R_1 : attributes in X^+ (i.e., $\text{Attr}(R_1) = X^+$).
 - * R_2 : attributes in X plus attributes NOT in X^+ (i.e., $\text{Attr}(R_2) = X \cup (\text{Attr}(R) - X^+)$).
 3. Recursively check and decompose R_1 and R_2 until all resultant tables are in BCNF.
- **Projection of Closures:** To check R_i after decomposition, compute closures on the original table R and *project* them onto R_i (by removing attributes not in $\text{Attr}(R_i)$).

9.1.5 Properties of BCNF

- **Good:** No update/deletion/insertion anomalies; minimal redundancy.
- **Lossless Join:** A BCNF decomposition guarantees a lossless join (the original table R can be reconstructed from $R_1 \bowtie R_2$) if the common attributes $\text{Attr}(R_1) \cap \text{Attr}(R_2)$ constitute a superkey of R_1 or R_2 . The BCNF algorithm ensures this.
- **Bad:** Dependencies may **not** be preserved (some FDs from the original table cannot be derived from the FDs on the decomposed tables).

9.2 Third Normal Form (3NF)

9.2.1 3NF Definition

- A table R satisfies 3NF if and only if for every non-trivial and decomposed FD $X \rightarrow A$ that holds on R , **at least one** of the following conditions is met:
 1. X is a **superkey** of R .
 2. A is a **prime attribute** of R (i.e., A appears in some key).

- **Intuition:** 3NF is more lenient than BCNF, allowing dependencies on non-superkeys (X) if those dependencies only determine attributes that are themselves part of a key (A).

9.2.2 BCNF vs. 3NF

- $\text{BCNF} \implies \text{3NF}$, but not vice versa.
- 3NF may still suffer from update/deletion anomalies in rare cases where a non-prime attribute depends transitively on only a partial key (but 3NF eliminates most redundancies).

9.2.3 3NF Check (Using Keys and Prime Attributes)

- Derive all keys (and thus prime attributes) for R .
- Check every given FD $X \rightarrow A$ against the two 3NF conditions.

9.2.4 3NF Decomposition (Synthesis Algorithm)

- **Goal:** Decompose R into smaller tables that are all in 3NF and guarantee dependency preservation.
- **Algorithm:**
 1. Derive a **minimal basis** M of the original FDs.
 2. Combine FDs in M that share the same left hand side (X).
 3. Create a table for each resulting FD $X \rightarrow Y$ (table attributes: $X \cup Y$).
 4. **Lossless Check:** If none of the created tables contains a key of the original table R , create an additional table R_{key} containing any key of R . (This ensures lossless join).
 5. Remove subsumed tables (tables whose attributes are a subset of another table's attributes).

9.2.5 Minimal Basis

A set of FDs M is a minimal basis for a set S if:

1. M and S are equivalent (every FD in M can be derived from S and vice versa).
2. Every FD in M is non-trivial and decomposed.
3. No FD in M is redundant (cannot be derived from the rest of M).
4. For any FD in M , removing any attribute from its left hand side means the resulting (shorter) FD cannot be derived from S .

9.2.6 Properties of 3NF

- **Good:** Lossless join property, and **preserves all FDs** (unlike BCNF).
- **Decision:** Prefer BCNF if a lossless, dependency-preserving decomposition exists. Otherwise, choose 3NF as it guarantees dependency preservation.